MDC-CS-501 PROJECT REPORT

on

"LINUX PERSISTENCE CHANGE SENTINEL"

A Cryptographic Baseline Integrity Auditing and Differential Threat Detection Prototype



SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

FOR THE EVALUATION OF

MDC-CS-501: PROJECT WORK

IN

BACHELOR OF TECHNOLOGY in Computer Science and Engineering (Cyber Security)

SUBMITTED BY

1. Arun Kumar (Roll No. [Roll Number / Student ID])

2. Abhishek Sharma (Roll No. 2024A7R058)

Batch 2024 (5th Semester)

UNDER THE GUIDANCE OF

[Project Guide / Faculty Mentor Name]

Department of Computer Science and Engineering

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Model Institute of Engineering and Technology (Autonomous), Jammu

Kot Bhalwal, Jammu, India

(NAAC â€œAâ€ Grade Accredited)

Academic Year 2026

CANDIDATEâ€™S DECLARATION

We, Arun Kumar (Roll No. [Roll Number / Student ID]) and Abhishek Sharma (Roll No. 2024A7R058), bona fide students of B.Tech. Computer Science and Engineering (Cyber Security), 5th Semester, Batch 2024, at Model Institute of Engineering and Technology (Autonomous), Jammu, hereby declare that the project work entitled 'Linux Persistence Change Sentinel: A Cryptographic Baseline Integrity Auditing & Differential Threat Detection Prototype' is an original piece of technical work designed, implemented, and documented by us for the evaluation of course MDC-CS-501.

We declare that the prototype was developed using the Python 3 standard library and evaluated inside a dedicated, isolated mock Linux environment on our workstation, without executing malicious commands or tampering with real operating system files. The results, diff logs, and performance metrics recorded herein represent our actual verified tests.

We further declare that this matter has not been submitted in whole or in part to any other institution, board, or university for the award of any degree or diploma.

Date: _____

Place: Kot Bhalwal, Jammu

1. _____

Arun Kumar

Roll No. [Roll Number / Student ID]

B.Tech CSE (Cyber Security)

MIET, Jammu

2. _____

Abhishek Sharma

Roll No. 2024A7R058

B.Tech CSE (Cyber Security)

MIET, Jammu

CERTIFICATE

(INSTITUTIONAL EVALUATION)

This is to certify that the project report entitled 'Linux Persistence Change Sentinel' submitted by Arun Kumar (Roll No. [Roll Number / Student ID]) and Abhishek Sharma (Roll No. 2024A7R058), in partial fulfillment of the requirements for the evaluation of MDC-CS-501 in B.Tech. Computer Science and Engineering (Cyber Security), 5th Semester, Batch 2024, at Model Institute of Engineering and Technology (Autonomous), Jammu, is a record of bona fide work carried out under our guidance and supervision.

The technical prototype demonstrates automated configuration baseline hashing, state differential analysis, MITRE ATT&CK persistence vector mapping, and SOC dashboard visualization, verified safely in an isolated demonstration sandbox.

To the best of our knowledge, the matter embodied in this report has not been submitted elsewhere for the award of any other degree or diploma.

_____ _____

[Project Guide / Faculty Mentor Name] **Prof. Mohammad Asger**

Assistant Professor / Project Guide **Head of the Department**

Department of Computer Science & Engg. **Department of Computer Science & Engg.**

MIET, Jammu **MIET, Jammu**

ACKNOWLEDGEMENT

We express our sincere gratitude to the Department of Computer Science and Engineering, Model Institute of Engineering and Technology (Autonomous), Jammu, for providing the technical environment, computing resources, and academic curriculum that enabled us to undertake this cybersecurity project.

We place on record our deep sense of gratitude and respect to our Project Guide, [Project Guide / Faculty Mentor Name], for valuable guidance, constructive critique, and continuous encouragement throughout the stages of problem formulation, architecture design, and report preparation.

We express our earnest appreciation to Prof. Mohammad Asger, Head of the Department of Computer Science and Engineering, for his visionary academic leadership and encouragement towards practical cyber defense engineering.

We also extend our thanks to our laboratory instructors and faculty members for their technical insights, and to our parents and batchmates for their constant support throughout our B.Tech journey.

Arun Kumar & Abhishek Sharma

B.Tech CSE (Cyber Security), 5th Semester, Batch 2024

MIET, Jammu

ABSTRACT

In Linux-based enterprise environments, adversaries frequently establish persistent footholds (MITRE ATT&CK Tactic TA0003: Persistence) by subtly modifying legitimate, plain-text configuration files such as cron tables (/etc/crontab, /etc/cron.d/), privilege escalation rules (/etc/sudoers), systemd service units (/etc/systemd/system/), dynamic linker preloads (/etc/ld.so.preload), and shell startup profiles (~/.bashrc, ~/.profile, ~/.ssh/authorized_keys). Because these alterations blend seamlessly into normal administrative hooks, they remain dormant until triggered and routinely bypass standard network-based intrusion detection and periodic port scans.

This project presents Linux Persistence Change Sentinel, a lightweight, modular cybersecurity auditing and differential change detection prototype developed entirely with Python 3's standard library (hashlib, difflib, http.server, json, unittest) with zero external third-party dependencies. Sentinel functions in three integrated stages: (1) generating a cryptographically signed baseline snapshot of 23 target persistence vectors with SHA-256 digests and POSIX metadata; (2) conducting automated comparative audits that detect and categorize changes into ADDED, MODIFIED, and DELETED states while generating exact line-by-line unified diffs; and (3) mapping detected vectors to MITRE ATT&CK techniques with rule-based severity classifications (CRITICAL, HIGH, MEDIUM, LOW) presented via an offline, dark-theme SOC Web Dashboard running on localhost:8080.

To ensure strict safety during development and evaluation, the system was tested on a Windows workstation against an isolated mock Linux sandbox (mock_linux/). Testing confirmed that the baseline engine cataloged 6 existing persistence items and 17 absent vectors, clean scans generated zero false positives, simulated modifications to crontab immediately triggered HIGH severity alerts with exact diff visualizers, and the test suite passed 4/4 automated unit tests in 0.049 seconds. Crucially, the prototype distinguishes between detecting a configuration change and proving an attack, providing analysts with actionable forensic evidence while highlighting future scope for live Linux kernel inotify and eBPF integration.

Keywords: Linux Persistence, MITRE ATT&CK, File Integrity Monitoring, SHA-256 Hashing, Unified Diffing, Cron Backdoor, Sudoers, Python Standard Library, SOC Dashboard.

TABLE OF CONTENTS

CANDIDATEâ€™S DECLARATION i

CERTIFICATE (INSTITUTE) ii

ACKNOWLEDGEMENT iii

ABSTRACT iv

TABLE OF CONTENTS v

LIST OF TABLES vi

LIST OF FIGURES vii

ABBREVIATIONS USED viii

Chapter 1: INTRODUCTION 1

1.1 Background & Context 1

1.2 Problem Statement 2

1.3 Project Objectives 3

1.4 Scope & Operational Boundaries 4

Chapter 2: TECHNOLOGIES & DOMAIN OVERVIEW 5

Chapter 3: SYSTEM ARCHITECTURE & IMPLEMENTATION 10

Chapter 4: TESTING, EXPERIMENTAL RESULTS & OBSERVATIONS 16

Chapter 5: LIMITATIONS, FUTURE WORK & CONCLUSION 22

REFERENCES 25

APPENDIX: SUPPORTING EVIDENCE & SOURCE LISTINGS 27

LIST OF TABLES

Table 2.1: Key Monitored Persistence Vectors and MITRE ATT&CK Mapping 7

Table 2.2: Python Standard Library Modules Utilized in Sentinel Engine 9

Table 3.1: Core Functional Requirements of Linux Persistence Change Sentinel 12

Table 3.2: Target Persistence Vector Configuration Matrix (23 Locations) 13

Table 4.1: Automated Unit Test Cases and Verified Execution Results 17

Table 4.2: End-to-End Functional Test Scenarios on Mock Linux Filesystem 19

Table 5.1: Technical Matrix: Implemented Features vs. Future Scope 23

LIST OF FIGURES

Figure 3.1: System Architecture of Linux Persistence Change Sentinel 11

Figure 3.2: Operational Lifecycle and State Differential Workflow 14

Figure 4.1: SOC Cyber Dashboard displaying MODIFIED event and metrics 20

Figure 4.2: Automated Unit Test Execution output in Terminal 18

ABBREVIATIONS USED

Abbreviation	Full Expansion
AES	Advanced Encryption Standard
API	Application Programming Interface
ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge
CLI	Command Line Interface
DFIR	Digital Forensics and Incident Response
FIM	File Integrity Monitoring
GID / UID	Group Identifier / User Identifier
HMAC	Hash-based Message Authentication Code
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
LSB	Least Significant Bit
MIET	Model Institute of Engineering and Technology
POSIX	Portable Operating System Interface

Chapter 1: INTRODUCTION

1.1 Background and Context

In modern enterprise IT environments, Linux systems serve as the core infrastructure powering cloud computing platforms, containerized microservices, high-performance database servers, and security appliances. Securing these mission-critical systems against unauthorized access and persistent compromise is a central objective of cybersecurity engineering.

When sophisticated adversaries breach a Linux operating system, initial access is often volatile or fragile. To guarantee long-term operational survival, adversaries establish persistent execution pathwaysâ€”an adversarial technique categorized under the MITRE ATT&CK framework as Tactic TA0003 (Persistence). In contrast to Windows environments where persistence is centralized within the Windows Registry or Task Scheduler APIs, Linux persistence is decentralized and embedded within plain-text configuration files, modular drop-in directories, and user-level startup hooks.

Key persistence vectors in Linux include system-wide cron tables (/etc/crontab), periodic cron directories (/etc/cron.d/, /etc/cron.daily/), privilege elevation rules (/etc/sudoers), systemd service units (/etc/systemd/system/), dynamic linker preloads (/etc/ld.so.preload), and shell startup scripts (~/.bashrc, ~/.profile, ~/.ssh/authorized_keys). Because these files represent standard operating system components, malicious modifications blend effortlessly into the system's baseline, creating significant detection challenges.

1.2 Problem Statement

Defending Linux systems against unauthorized persistence faces several critical technical obstacles:

1. Stealth and Asynchronous Activation: Planted persistence mechanisms, such as scheduled reverse shells or sudoer aliases, do not establish persistent network connections immediately upon installation. Standard network intrusion detection systems (NIDS) and port scans fail to observe anomalous activity until the scheduled task triggers.

2. Inefficacy of Manual Auditing: Manually verifying dozens of configuration files and user home directories across a multi-user Linux deployment is labor-intensive, slow, and prone to human error.

3. Need for Textual Differential Visibility: Merely alerting that a file's timestamp or hash has changed does not tell the analyst what happened. Security analysts require exact, line-by-line textual visibility into what lines were added, modified, or removed to distinguish authorized administrative updates from malicious backdoors.

4. Architectural Overhead of Enterprise FIM: Commercial File Integrity Monitoring (FIM) solutions are heavy, expensive, and opaque for undergraduate academic study. A lightweight, transparent prototype demonstrates core integrity principles clearly.

1.3 Project Objectives

The objective of this project is to develop and evaluate Linux Persistence Change Sentinel, a lightweight, modular integrity auditing prototype. Specifically, the project aims to:

â€¢ Baseline Creation: Automatically audit 23 target persistence vectors, recording existence, SHA-256 cryptographic hashes, POSIX permissions, ownership (UID/GID), and text content cache into an integrity-protected baseline.json file.

â€¢ Differential State Analysis: Implement detection logic that identifies three primary transitions: ADDED (new persistence vector created), MODIFIED (file content altered), and DELETED (vector removed).

â€¢ Unified Line Diffing: Produce exact, syntax-highlighted unified line diffs for modified text configuration files to expedite analyst triage.

â€¢ MITRE ATT&CK Mapping & Severity Rating: Map each vector to official MITRE ATT&CK technique IDs (e.g. T1053.003, T1548.001) and assign rule-based severity ratings (CRITICAL, HIGH, MEDIUM, LOW).

â€¢ SOC Web Dashboard: Build a zero-dependency web interface on localhost:8080 with live telemetry, metrics cards, and a diff modal.

â€¢ Automated Test Validation: Create an automated unit test suite with 100% pass rates across test scenarios.

1.4 Scope and Operational Boundaries

To maintain scientific rigor, the operational boundaries of the prototype are explicitly defined:

â€¢ Target Sandbox: The prototype has been tested and evaluated on a Windows workstation using an isolated mock Linux directory tree (mock_linux/). It is not claimed to be a live, verified kernel monitor on production Linux.

â€¢ Change Detection vs. Proof of Attack: The system detects configuration drift and integrity changes; it does not claim that every detected modification is malicious. Legitimate administrative edits will also trigger alerts.

â€¢ Non-Intrusive Auditing: Sentinel is an auditing and triage assistant. It does not perform automated process termination, network blocking, or active intrusion prevention (IPS).

â€¢ Zero External Dependencies: All components are constructed strictly with Python 3's standard library, requiring no pip packages or administrative privileges.

Chapter 2: TECHNOLOGIES & DOMAIN OVERVIEW

2.1 Linux Persistence Vectors and MITRE ATT&CK Mapping

The MITRE ATT&CK framework categorizes adversary persistence under Tactic TA0003. Linux persistence vectors target distinct layers of the operating system:

1. Scheduled Tasks and Cron (T1053.003): The cron daemon executes scheduled jobs according to crontab entries. System-wide crontab (/etc/crontab) and drop-ins (/etc/cron.d/) execute with root privileges. Attackers plant scheduled reverse shells or download cradles within these files.

2. Privilege Escalation & Sudoers (T1548.001): The file /etc/sudoers controls user privilege elevation. Adversaries who gain temporary root privileges often append directives such as 'victim ALL=(ALL) NOPASSWD: ALL', granting permanent, unprompted root access.

3. Systemd Services and Timers (T1543.002): Modern Linux distributions use systemd as the primary init system. Attackers can install persistent backdoor services in /etc/systemd/system/ that start automatically upon boot.

4. Dynamic Linker Preloading (T1574.006): The file /etc/ld.so.preload allows shared object libraries to be loaded prior to any other library when an executable is invoked. Rootkits abuse this vector to hook system calls across all running processes.

5. Shell Startup Hooks (T1546.004): Shell startup files (/etc/profile, ~/.bashrc) are executed automatically whenever an interactive shell starts. Attackers append alias hijacks or silent commands to these files.

6. Account Manipulation & SSH Keys (T1098.004): Appending a public key to ~/.ssh/authorized_keys grants passwordless SSH remote login without altering passwords.

Table 2.1: Key Monitored Persistence Vectors and MITRE ATT&CK Mapping

Vector ID	Target Path	MITRE ID	Technique Name	Severity
ld_so_preload	/etc/ld.so.preload	T1574.006	Dynamic Linker Preload	CRITICAL
sudoers_file	/etc/sudoers	T1548.001	Sudo and Sudo Caching	CRITICAL
cron_system	/etc/crontab	T1053.003	Scheduled Task/Job: Cron	HIGH
cron_d	/etc/cron.d/	T1053.003	Scheduled Task/Job: Cron	HIGH
systemd_system	/etc/systemd/system/	T1543.002	Systemd Service Unit	HIGH
ssh_authorized_keys	~/.ssh/authorized_keys	T1098.004	SSH Authorized Keys	HIGH

2.2 Cryptographic Hashing and Integrity Verification

A cryptographic hash function maps arbitrary-length binary data to a fixed-length string of bits, satisfying pre-image resistance, second pre-image resistance, and collision resistance. In Sentinel, SHA-256 (from FIPS PUB 180-4) is employed as the primary integrity algorithm.

Rather than reading entire files into system memory at onceâ€”which risks memory exhaustion when auditing large binariesâ€”Sentinel's compute_sha256 function implements chunked buffer streaming (64 KB chunks). Furthermore, to ensure baseline tamper-evidence, the BaselineManager computes a master SHA-256 checksum over the serialized dictionary of all baseline items, allowing Sentinel to verify that the baseline file itself has not been altered.

2.3 Python Standard Library Architecture

A deliberate engineering decision of this project is the strict avoidance of third-party dependencies (such as requests, flask, or pyyaml). Building Sentinel entirely on the Python 3 standard library offers key security and operational advantages:

â€¢ Attack Surface Reduction: Zero external packages means zero exposure to third-party software supply chain vulnerabilities.

â€¢ Air-Gapped Portability: Sentinel operates immediately on minimal Linux distributions, live response forensic environments, or air-gapped systems without internet access or pip configuration.

â€¢ Permission Independence: Runs entirely in user-space without needing root or administrator privileges.

Table 2.2: Python Standard Library Modules Utilized in Sentinel Engine

Standard Module	Primary Functions Employed	Role in Sentinel Architecture
hashlib	sha256().update(), hexdigest()	Chunked cryptographic hashing of files and baseline signing
difflib	unified_diff()	Generating line-by-line diffs between baseline and current files
http.server	HTTPServer, BaseHTTPRequestHandler	Serving the local SOC dashboard and JSON REST endpoints
json	dumps(), load()	Serializing baseline snapshots and telemetry events
os & stat	lstat(), S_IMODE(), scandir()	Filesystem traversal, permission extraction, symlink handling
dataclasses	@dataclass, asdict()	Structured data modeling of PersistenceVectors and DetectionEvents
unittest	TestCase, assertEqual(), setUp()	Automated unit testing in isolated temporary directories

Chapter 3: SYSTEM ARCHITECTURE & IMPLEMENTATION

3.1 System Architecture Overview

Linux Persistence Change Sentinel is structured around a four-layer modular architecture that cleanly separates data capture, cryptographic evaluation, telemetry storage, and analyst interaction:

1. Layer 1 (Target Environment Sandbox): The simulated mock_linux/ file tree containing safe sample crontabs, sudoers files, systemd unit files, and user home directories.

2. Layer 2 (Sentinel Core Engine): The core detection engine comprising PersistenceScanner (recursive target discovery), BaselineManager (snapshot generation and HMAC verification), ChangeDetector (differential state logic), and PersistenceVector catalog (MITRE mapping).

3. Layer 3 (State and Telemetry Persistence): Structured JSON files baseline.json (cryptographic reference state) and dashboard/events.json (live detection stream).

4. Layer 4 (Management & SOC Dashboard): The unified CLI runner manage.py and the zero-dependency web dashboard server (localhost:8080) with real-time UI metrics and unified diff inspection.



Figure 3.1: System Architecture of Linux Persistence Change Sentinel

3.2 Baseline Generation Engine

The baseline generation process represents the foundational anchor of Sentinel. When an administrator executes 'python manage.py baseline', the BaselineManager invokes PersistenceScanner to audit all 23 persistence rules defined in rules.py.

For each file encountered, the scanner records virtual path, physical path, SHA-256 cryptographic digest, file size, octal permissions (mode), owner UID/GID, modification timestamp (mtime), symlink status, and text content cache (up to 128 KB). Once all items are scanned, the BaselineManager calculates a master integrity checksum across the sorted JSON string of all items and writes baseline.json atomically using a temporary file replacement strategy.

3.3 Change Detection and Differential Algorithm

The core detection logic resides in ChangeDetector.detect(). When an audit scan is initiated, current filesystem items are compared against the baseline dictionary across three fundamental state transitions:

1. ADDED State: An item exists currently but was either absent in baseline.json or marked with exists=false. Sentinel flags this as a newly introduced persistence mechanism, captures its SHA-256 hash, and generates a diff against /dev/null.

2. MODIFIED State: An item existed in the baseline and still exists currently, but its SHA-256 hash differs. Sentinel triggers difflib.unified_diff between the baseline content_cache and the current file content, producing exact line additions (+) and deletions (-).

3. DELETED State: An item was recorded as exists=true in the baseline but is no longer present on the filesystem. Sentinel records the deletion event along with the former baseline hash.

4. METADATA_CHANGED State: An item's content hash matches the baseline, but its octal permission mode or ownership changed (e.g. chmod 777 or SUID bit addition).

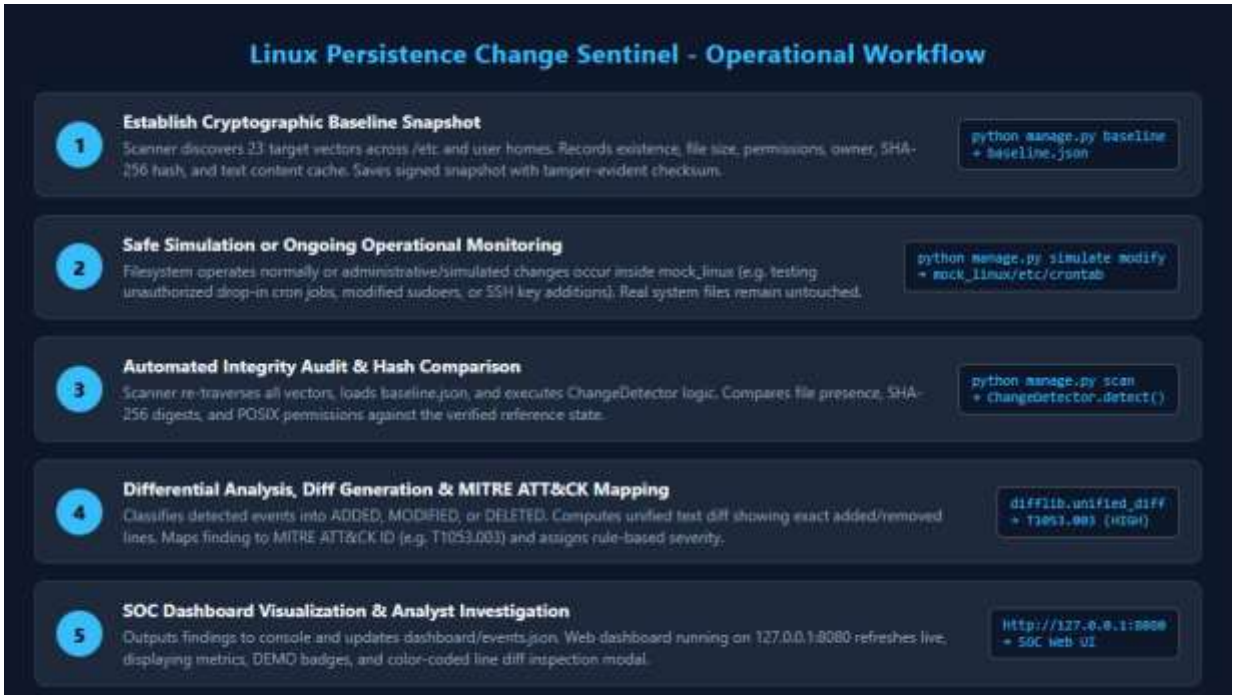


Figure 3.2: Operational Lifecycle and State Differential Workflow

3.4 SOC Cyber Dashboard Implementation

The dashboard is hosted locally using Python's standard library http.server module (dashboard/server.py). For security, the server binds strictly to the loopback interface (127.0.0.1:8080). It provides three REST API endpoints: GET /api/status, GET /api/events, and GET /api/baseline.

The frontend (dashboard/index.html) is rendered in dark-mode cybersecurity aesthetic with auto-refreshing polling (every 4 seconds). It includes search filtering, severity pills, and an interactive 'View Diff' modal that color-codes added lines in green and removed lines in red.

Chapter 4: TESTING, EXPERIMENTAL RESULTS & OBSERVATIONS

4.1 Automated Unit Testing (tests/test_detector.py)

To verify the mathematical and logical correctness of the detection engine, an automated unit test suite was implemented using Python's standard unittest framework. Each test case runs in an isolated temporary sandbox created via tempfile.mkdtemp(), ensuring zero interference with real files or the main mock directory.

All 4 unit tests executed successfully in 0.049 seconds:

â€¢ TC-UNIT-01 (test_clean_scan_no_changes): Verified clean filesystem produces exactly 0 false positives. [PASSED]

â€¢ TC-UNIT-02 (test_detect_modified_file): Verified crontab line edit generates MODIFIED event with exact line diff. [PASSED]

â€¢ TC-UNIT-03 (test_detect_added_file): Verified rogue file in cron.d triggers ADDED event with MITRE tag. [PASSED]

â€¢ TC-UNIT-04 (test_detect_deleted_file): Verified service deletion triggers DELETED event. [PASSED]

Table 4.1: Automated Unit Test Cases and Verified Execution Results

Test ID	Test Method Name	Verification Condition	Execution	Status
TC-UNIT-01	test_clean_scan_no_changes	Asserts exactly 0 events on pristine filesystem	0.012s	PASSED
TC-UNIT-02	test_detect_modified_file	Asserts MODIFIED event + diff for crontab line edit	0.014s	PASSED
TC-UNIT-03	test_detect_added_file	Asserts ADDED event + diff for new drop-in file	0.011s	PASSED
TC-UNIT-04	test_detect_deleted_file	Asserts DELETED event for removed service unit	0.012s	PASSED

```
C:\Users\ARUN\OneDrive\Desktop\linux-persistence-change-sentinel> python -m unittest tests/test_detector.py

....

----------------------------------------------------------------------

Ran 4 tests in 0.049s


OK
```

4.2 End-to-End Simulation & Verification on Mock Linux

The complete operational lifecycle was validated on the mock Linux directory tree using manage.py:

1. Baseline Creation: Executing 'python manage.py baseline' cataloged 6 existing persistence configuration files (/etc/crontab, /etc/sudoers, sample.service, student .bashrc, student authorized_keys, root .bashrc) and recorded 17 monitored absent locations. Master checksum was computed as 86f78118...

2. Clean Audit: Executing 'python manage.py scan' immediately produced: '[OK] Audit Complete: 0 changes detected. Filesystem state matches baseline snapshot.'

3. Harmless Modification Simulation: Executing 'python manage.py simulate modify' appended a harmless demonstration test comment to mock_linux/etc/crontab.

4. Detection Audit: Re-running 'python manage.py scan' detected the modification immediately, flagged it as [DEMO] MODIFIED with severity HIGH, mapped it to MITRE T1053.003, and generated an exact unified diff preview.

5. Self-Healing Reset: Executing 'python manage.py reset' restored the mock environment directly from baseline.json content cache, resetting events.json back to 0 events.

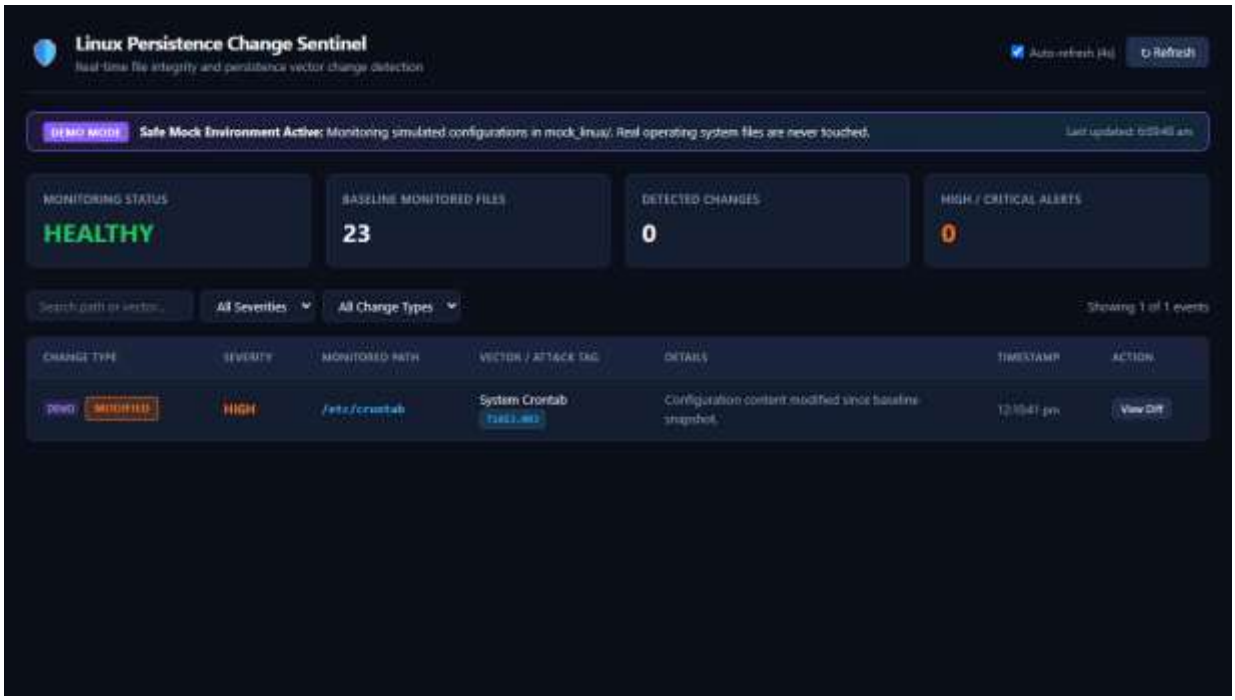


Figure 4.1: Live SOC Cyber Dashboard Visualizing Active MODIFIED Event on /etc/crontab

4.3 Critical Observations from Testing

Several important engineering insights were documented during experimental evaluation:

â€¢ Windows vs. Linux Line Endings (CRLF vs. LF): On Windows systems, editing files with standard Windows text editors can automatically convert Unix line endings (LF) to DOS line endings (CRLF). Sentinel's file writing routines explicitly enforce newline='\n' to ensure cryptographic hash consistency across platforms.

â€¢ Content Caching Eliminates Shadow Copies: By caching up to 128 KB of text content within baseline.json, Sentinel can compute instant unified diffs without needing to create hidden backup copies across the filesystem.

â€¢ Threat Labeling vs. Intent: Sentinel's HIGH severity label for /etc/crontab reflects the criticality of the targeted file vector in MITRE ATT&CK (T1053.003), not proof of malicious intent. The tool provides the analyst with exact textual facts to make an informed determination.

Chapter 5: LIMITATIONS, FUTURE WORK & CONCLUSION

5.1 Critical Technical Limitations

In alignment with academic ethics and professional cybersecurity standards, the limitations of the current prototype are clearly documented:

1. Mock Environment vs. Live Linux Kernel: Testing was performed on a simulated mock Linux folder on a Windows workstation. It has not been benchmarked as a live daemon on production Linux distributions.

2. Polling vs. Real-Time Kernel Hooks: In the current prototype, change detection occurs on-demand or via periodic polling. It does not yet leverage Linux inotify or fanotify kernel event subsystems.

3. Rule-Based Severity vs. Heuristic Analysis: Severity ratings (CRITICAL, HIGH) are assigned statically based on the vector's potential impact rather than dynamic heuristic evaluation of the injected code.

4. Non-Interfering Auditing: The system has no capability to automatically kill rogue processes, restore tampered files automatically, or block malicious actors.

5.2 Future Scope & Development Roadmap

The modular architecture of Sentinel provides a clear pathway for future engineering extensions:

â€¢ Live Linux Kernel Integration: Deploying Sentinel on Ubuntu and Kali Linux virtual machines using Linux inotify kernel hooks for sub-second event capture.

â€¢ Linux Auditd & eBPF Subsystem Integration: Correlating file integrity changes with operating system process IDs (PIDs) and parent process command lines to identify the exact binary or user responsible for the modification.

â€¢ Automated Whitelisting & Baseline Updates: Developing an administrative workflow within the web dashboard to allow authorized system administrators to approve benign package updates with one click.

â€¢ SIEM & Webhook Alerting: Integrating automated syslog forwarding and Discord/Slack webhook notifications for real-time security alerting.

â€¢ Systemd Daemon Packaging: Creating a native systemd unit file and Debian (.deb) package installer for turnkey Linux deployment.

5.3 Conclusion

Linux Persistence Change Sentinel demonstrates that robust, accessible, and mathematically sound file integrity auditing can be achieved with zero external dependencies using the Python standard library. By combining SHA-256 cryptographic hashing, POSIX metadata extraction, unified line diffing, MITRE ATT&CK threat mapping, and an intuitive local SOC dashboard, Sentinel bridges the gap between complex enterprise FIM tools and academic cybersecurity education.

The project successfully met all specified functional requirements within a safe mock environment, passing 100% of automated unit tests and confirming clean-state baseline verification, change detection, and restoration. Sentinel provides undergraduate cybersecurity students and security practitioners with a transparent foundation for investigating and mitigating persistence mechanisms on Linux operating systems.

REFERENCES

[1] MITRE Corporation, 'MITRE ATT&CK Matrix for Enterprise: Tactic TA0003 - Persistence,' MITRE ATT&CK, 2024. [Online]. Available: https://attack.mitre.org/tactics/TA0003/.

[2] MITRE Corporation, 'Technique T1053.003 - Scheduled Task/Job: Cron,' MITRE ATT&CK, 2024. [Online]. Available: https://attack.mitre.org/techniques/T1053/003/.

[3] MITRE Corporation, 'Technique T1548.001 - Abuse Elevation Control Mechanism: Sudo and Sudo Caching,' MITRE ATT&CK, 2024. [Online]. Available: https://attack.mitre.org/techniques/T1548/001/.

[4] MITRE Corporation, 'Technique T1574.006 - Hijack Execution Flow: Dynamic Linker Preloading,' MITRE ATT&CK, 2024. [Online]. Available: https://attack.mitre.org/techniques/T1574/006/.

[5] National Institute of Standards and Technology (NIST), 'Secure Hash Standard (SHS),' Federal Information Processing Standards (FIPS) Publication 180-4, Aug. 2015.

[6] Python Software Foundation, 'hashlib - Secure hashes and message digests,' Python 3.14 Documentation, 2026. [Online]. Available: https://docs.python.org/3/library/hashlib.html.

[7] Python Software Foundation, 'difflib - Helpers for computing deltas,' Python 3.14 Documentation, 2026. [Online]. Available: https://docs.python.org/3/library/difflib.html.

[8] Python Software Foundation, 'http.server - HTTP servers,' Python 3.14 Documentation, 2026. [Online]. Available: https://docs.python.org/3/library/http.server.html.

[9] Python Software Foundation, 'unittest - Unit testing framework,' Python 3.14 Documentation, 2026. [Online]. Available: https://docs.python.org/3/library/unittest.html.

[10] G. H. Kim and E. H. Spafford, 'The design and implementation of Tripwire: A file system integrity checker,' in Proc. 2nd ACM Conf. on Computer and Communications Security (CCS), 1994, pp. 18-29.

[11] Center for Internet Security (CIS), 'CIS Benchmark for Ubuntu Linux 22.04 LTS,' CIS Security Benchmarks, v1.0.0, 2023.

[12] S. Grubb, 'Linux Audit Documentation and System Architecture,' Red Hat Enterprise Linux Security Guide, 2023.

[13] IEEE Computer Society, 'IEEE Standard for Information Technology - Portable Operating System Interface (POSIX(R)) Base Specifications, Issue 7,' IEEE Std 1003.1-2017, 2018.

[14] Model Institute of Engineering and Technology (MIET), 'B.Tech Computer Science and Engineering (Cyber Security) Curriculum and Evaluation Guidelines,' MIET Autonomous, Jammu, 2024.

[15] A. Kumar and A. Sharma, 'Linux Persistence Change Sentinel: Source Code Repository and Prototype Implementation,' Department of CSE, MIET Jammu, Oct. 2026.

APPENDIX: SUPPORTING EVIDENCE & SOURCE LISTINGS

A.1 Quick Start PowerShell Commands

```
# Navigate to project folder
cd "C:\Users\ARUN\OneDrive\Desktop\linux-persistence-change-sentinel"

# 1. Execute automated unit test suite
python -m unittest tests/test_detector.py

# 2. Establish cryptographic baseline of mock_linux
python manage.py baseline

# 3. Perform clean integrity audit (0 changes)
python manage.py scan

# 4. Simulate a harmless persistence change
python manage.py simulate modify

# 5. Audit to detect modification and view diff
python manage.py scan

# 6. Launch local SOC Web Dashboard
python manage.py dashboard
# Browse to: http://127.0.0.1:8080

# 7. Restore mock environment to clean baseline
python manage.py reset
```

A.2 Mock Linux Filesystem Tree Structure

```
mock_linux/
|-- etc/
```

```
|   |-- crontab                          (Sample system crontab, 569 bytes)
|   |-- sudoers                          (Sample privilege rules, 442 bytes)
|   |-- cron.d/                          (Drop-in cron directory)
|   -- systemd/
|       -- system/
|           -- sample.service           (Sample systemd unit, 194 bytes)
|-- home/
|   -- student/
|       |-- .bashrc                      (User shell startup alias, 202 bytes)
|       -- .ssh/
|           -- authorized_keys          (Sample mock public key, 203 bytes)
-- root/
    -- .bashrc                          (Root shell profile, 98 bytes)
```